```
# Import Required Libraries

import pandas as pd

import matplotlib.pyplot as plt

import seaborn as sns

import numpy as np

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler

from sklearn.metrics import classification_report, accuracy_score, confusion_matrix

from sklearn.ensemble import RandomForestClassifier

from sklearn.tree import DecisionTreeClassifier

from sklearn.naive_bayes import GaussianNB

from sklearn.neighbors import KNeighborsClassifier

import pickle


# Load the Augmented Dataset

df = pd.read_csv("augmented_animal_disease_prediction.csv")


# Identify target column

target_col = [col for col in df.columns if 'Disease_Prediction' in col][0]


# Separate Features and Target Correctly

X = df.drop(columns=[target_col])

y = df[target_col]


print("Unique values in y:", y.unique())
```

```
# Encode only the feature columns if necessary

categorical_cols = X.select_dtypes(include=['object']).columns.tolist()

X_encoded = pd.get_dummies(X, columns=categorical_cols, drop_first=True)


# Normalize numeric features

scaler = StandardScaler()

X_scaled = scaler.fit_transform(X_encoded)


#  No Feature Selection — Keep all scaled features


# Train-Test Split

X_train, X_test, y_train, y_test = train_test_split(

    X_scaled, y, test_size=0.2, stratify=y, random_state=42

)


# Define Models

models = {

    "Decision Tree": DecisionTreeClassifier(max_depth=10, min_samples_split=4, random_state=42),

    "Random Forest": RandomForestClassifier(n_estimators=100, max_depth=10, min_samples_split=4,
random_state=42),

    "Naive Bayes": GaussianNB(),

    "KNN": KNeighborsClassifier(n_neighbors=5, weights='uniform')

}


# Train, Predict and Evaluate
```

```
performance_metrics = []
conf_matrices = {}

for name, model in models.items():
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)

    acc = accuracy_score(y_test, y_pred)
    report = classification_report(y_test, y_pred, output_dict=True)

    print(f"\n{name} - Classification Report")
    print(classification_report(y_test, y_pred))

    # Save metrics
    performance_metrics.append({
        "Model": name,
        "Accuracy": round(acc * 100, 2),
        "Precision": round(report["weighted avg"]["precision"] * 100, 2),
        "Recall": round(report["weighted avg"]["recall"] * 100, 2),
        "F1-Score": round(report["weighted avg"]["f1-score"] * 100, 2)
    })

    # Save Confusion Matrix
    conf_matrices[name] = confusion_matrix(y_test, y_pred)
```

```
# Create DataFrame for Metrics
metrics_df = pd.DataFrame(performance_metrics)

#  Plot Performance Comparison
metrics = ["Accuracy", "Precision", "Recall", "F1-Score"]
colors = ["skyblue", "orange", "lightgreen", "violet"]

for metric, color in zip(metrics, colors):
    plt.figure(figsize=(8, 5))
    plt.bar(metrics_df["Model"], metrics_df[metric], color=color, edgecolor='black')
    plt.title(f'{metric} Comparison of ML Models')
    plt.xlabel('Model')
    plt.ylabel(f'{metric} (%)')
    plt.ylim(0, 110)
    plt.grid(axis='y', linestyle='--', linewidth=0.5)
    plt.tight_layout()
    plt.show()

# Plot Confusion Matrices
for name, cm in conf_matrices.items():
    plt.figure(figsize=(6, 4))
    sns.heatmap(cm, annot=True, fmt='d', cmap="Blues")
    plt.title(f"{name} - Confusion Matrix")
    plt.xlabel("Predicted")
    plt.ylabel("Actual")
```

```
    plt.tight_layout()
    plt.show()

#  Final Summary Table
print("\n=== Summary Metrics ===")
print(metrics_df)

#  Save Best Model (highest Accuracy)
best_model_name = metrics_df.loc[metrics_df["Accuracy"].idxmax()]["Model"]
best_model = models[best_model_name]

# Retrain best model on full scaled data
best_model.fit(X_scaled, y)



# Save the fitted scaler
with open("scaler.pkl", "wb") as f:
    pickle.dump(scaler, f)

# Save the feature columns after dummies
pd.Series(X_encoded.columns).to_csv("model_features.csv", index=False, header=False)



# Save the model
with open(f"{best_model_name.replace(' ', '_').lower()}_best_model.pkl", "wb") as file:
```

```
    pickle.dump(best_model, file)


print(f"\n Best model '{best_model_name}' saved as '{best_model_name.replace(' ',
'_').lower()}_best_model.pkl'")


# Save the metrics DataFrame

metrics_df.to_csv("model_performance_summary.csv", index=False)

print("\n Model performance summary saved as 'model_performance_summary.csv'")
```